Master Architectural Index

Deterministic Admissibility Engineering (DAE)

Sovereign Catalog, Invariant Mapping, and Zenodo DOI Archival Anchor for the Master Specification Suite

Jesse Tuohy (Principal Invariant Architect) | **ORCID:** 0009-0008-4661-1540
Arcstone Adaptive Science Systems

Publication Date: October 2026

Primary Suite Zenodo DOI	`10.5281/zenodo.22665852`
Document Zenodo DOI	`10.5281/zenodo.23371581`
Specification Collection	ARC-ERR-2026-001, ARC-SPEC-2026-EDU-01–09, EXEC-01
Target Baseline	Phase 7 Realization (`v1.3.0-exec`)
Repository Core	`arcstone-continuity-core/docs/academic/`
Governing Invariants	$C_{\text{ops}} = 0$ | $\tau_{\text{override}} \leq 11.99\text{ms}$ | Heap Allocation = 0 bytes

---

1. Master Architectural Index Overview

This document serves as the primary citation anchor, architectural map, and verification index for the complete 11-document Deterministic Admissibility Engineering (DAE) publication suite. DAE replaces permissive execution models across computer science education, system kernels, and mechatronic robotics with pre-execution admissibility validation governed by a single axiom:

AXIOM OF ADMISSIBILITY: Existence $\neq$ Permission

2. Complete Document Catalog & DOI Invariant Map

Table 1: DAE Academic Specification Suite Index

Document ID	Title & Primary Domain	Governing Invariant / Focus
ERR-2026-001	Master Canonical Errata & Disambiguation	PWC Canonical Lock (POSIX 12)
EDU-01	Foundations of DAE	$C_{\text{ops}} = 0$ | $\tau_{\text{override}} \leq 11.99\text{ms}$
EDU-02	Laboratory Verification Harness	Zero Heap (0 bytes), WCET $\leq$ 11.99ms
EDU-03	Industrial Accreditation & ABET	ABET SO 1–7 Mapping ($A_{\text{drag}} = 0$)
EDU-04	Autonomic Courseware Synthesis	AI Grounding Schema ($H_{\text{prompt}} = 0$)
EDU-05	Formal Verification & Coq Proofs	Mechanical Soundness ($\mathbb{P}_{\text{err}} = 0$)
EDU-06	Cross-Domain Substrates	Physical Jitter ($\Delta J \leq 100\text{ns}$)
EDU-07	Pedagogical Cryptography & ZK	Zero Code Leakage ($\mathcal{K}_{\text{leak}} = 0$ bits)
EDU-08	Autonomous Agentic Pedagogy	Zero Pedagogical Drift ($\Delta P = 0$)
EDU-09	Universal Systems Transduction	Cross-Disciplinary Entropic Drift ($\Delta E = 0$)
EXEC-01	Dual Launch Execution Protocol	Zenodo Deposit & GitHub Release

3. Archival & Citation Instructions

To cite this complete specification suite in academic publications or technical specs:

> *Tuohy, J. (2026). Master Architectural Index: Deterministic Admissibility Engineering (DAE) [ARC-ERR-2026-001, ARC-SPEC-2026-EDU-01 through EDU-09, ARC-SPEC-2026-EXEC-01]. Arcstone Adaptive Science Systems. Zenodo. DOI: 10.5281/zenodo.23371581*

ARCHITECTURAL INDEX COMPLETE: 00_Master_Architectural_Index_DAE
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

ARC-SPEC-2026-EDU-01: Foundations of Deterministic Admissibility Engineering (DAE)

A Mathematical, Architectural, and Pedagogical Specification for Post-Permissive Computing

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants: $C_{\text{ops}} = 0 \quad | \quad \tau_{\text{override}} \leq 11.99\text{ms}$
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Foundational Axioms

Modern computing education is built on an insecure foundation: permissive execution. From introductory C/Python courses to advanced kernel design, students are taught to write systems that allocate resources, instantiate runtime contexts, and execute instructions first, relying on post-hoc error handling, exception handlers, or sandboxes to catch failures. In high-throughput, autonomous, or edge environments, this paradigm leads to state drift, non-deterministic latency spikes, and structural vulnerability.

This specification formalizes Deterministic Admissibility Engineering (DAE)—a pedagogical and architectural framework built on a single governing axiom:

AXIOM OF ADMISSIBILITY: Existence $\neq$ Permission

Under DAE, no instruction, neural network inference, or physical actuator movement may execute until its state vector passes a pre-execution admissibility gate at the kernel or hardware boundary.

Model Paradigm Comparison

PERMISSIVE MODEL (Legacy CS Education):

[State Change Trigger] $\longrightarrow$ [Allocate Heap / Execute] $\longrightarrow$ [Intercept Error] (High Risk)

DETERMINISTIC ADMISSIBILITY MODEL (DAE Curriculum):

[State Change Trigger] $\longrightarrow$ [Evaluate Admissibility Gate] $\longrightarrow$ [Execute PASS] (Zero Entropy)
$\Downarrow$
$\longrightarrow$ [Short-Circuit FAIL/FREEZE] (POSIX 40 / Sub-12ms)

2 Theoretical & Mathematical Foundations

2.1 The 5D Dynamic State Vector Space (Ψ_{Entity})

To teach students how to reason about complex system state without relying on unbounded boolean flags, DAE maps all thread, process, and entity dynamics into a continuous 5-dimensional coordinate

space:

$$\mathbf{\Psi}_{\text{Entity}}(t) = \begin{pmatrix} \mathcal{T} \\ \mathcal{A} \\ \mathcal{S} \\ \mathcal{I} \\ \mathcal{C} \end{pmatrix} = \begin{pmatrix} \text{Temporal Topology} \\ \text{Agency Vector} \\ \text{Substrate Coupling} \\ \text{Scale of Impact} \\ \text{Constraint Density} \end{pmatrix} \tag{1}$$

Where:

- $\mathcal{T}$ (**Temporal Topology**): Quantifies local clock skew and tick drift relative to hardware invariants.
- $\mathcal{A}$ (**Agency Vector**): Bounded privilege tier $[0.0, 1.0]$ governing state mutation authority.
- $\mathcal{S}$ (**Substrate Coupling**): Direct hardware/memory mapping depth (e.g., bare-metal registers vs. user-space IPC).
- $\mathcal{I}$ (**Scale of Impact**): Potential blast radius of state changes across downstream nodes.
- $\mathcal{C}$ (**Constraint Density**): Total active invariant rules enforced on the executing thread.

Systemic tension (V_a) and state drift velocity are computed via the temporal derivative:

$$V_a(t) = \frac{d\mathbf{\Psi}_{\text{Entity}}}{dt} = \begin{pmatrix} \dot{\mathcal{T}} & \dot{\mathcal{A}} & \dot{\mathcal{S}} & \dot{\mathcal{I}} & \dot{\mathcal{C}} \end{pmatrix}^T \tag{2}$$

Educational Insight: Students learn that unconstrained systems ($\mathcal{C} \to 0$) experience exponential authority expansion ($\dot{\mathcal{A}} \to \infty$), forcing state degradation unless counterbalanced by invariant clamping.

2.2 The Non-Collapsible Dominance Lattice Algebra

Traditional error handling uses arbitrary, branch-predicted `if-else` trees. DAE replaces these with a partially ordered set (poset) evaluation algebra that governs real-time execution:

$$\mathcal{L}_{\text{Dominance}} = \langle \{\text{FAIL}, \text{FREEZE}, \text{PWC}, \text{REFUSAL}, \text{PASS}\}, \succ \rangle \tag{3}$$

The evaluation chain is strictly hierarchical and non-collapsible:

$$\begin{aligned} \text{FAIL (POSIX 40)} \succ \text{FREEZE (POSIX 10)} \succ \text{PWC (POSIX 12)} \\ \succ \text{REFUSAL (POSIX 32)} \succ \text{PASS (POSIX 0)} \end{aligned} \tag{4}$$

Table 1: Lattice State & Microarchitectural Action Table

Mask	POSIX	System Behavior	Microarchitectural Action
FAIL	POSIX 40	Complete Execution Halt	Trigger Hardware NMI
FREEZE	POSIX 10	State Lock / Hold Position	Clamp Step Clocks to Zero
PWC	POSIX 12	Pending Worker Clearance	Hold Pending Out-of-Band Token
REFUSAL	POSIX 32	Wire-Speed Packet Drop	eBPF `XDP_DROP` Ring 0 Filter
PASS	POSIX 0	Admissible Execution	Zero-Copy Register Transfer

Mathematical Guarantee: For any combination of downstream inputs, the lattice operator $\mathcal{L}_{\text{eval}}(\mathbf{S})$ resolves to the supremum (least upper bound) of safety violations:

$$\mathcal{L}_{\text{eval}}(\mathbf{S}) = \sup_i \{s_i \in \mathbf{S}\} \tag{5}$$

If a single sensor or rule emits `FAIL`, the entire system resolves to `FAIL` regardless of neural network or optimizer confidence scores.

2.3 Kingman-Clamped Queuing & Latency Bounds

To prevent heap exhaustion and unbounded queueing delay (W_q), DAE incorporates Kingman's approximation to enforce hard utilization (ρ) bounds:

$$W_q \approx \left(\frac{\sigma_a^2 + \sigma_s^2}{2}\right)\left(\frac{\rho}{1-\rho}\right)\frac{1}{\mu} \tag{6}$$

As arrival rate approaches service capacity ($\rho \to 1.0$), latency W_q approaches infinity. DAE curricula teach students to clamp queues mathematically:

$$\text{Mode}(\rho) = \begin{cases} \text{NOMINAL (PASS)} & \text{for } \rho \leq 0.80 \\ \text{PWC (THROTTLE)} & \text{for } 0.80 < \rho < 0.95 \\ \text{REFUSAL (XDP_DROP)} & \text{for } \rho \geq 0.95 \end{cases} \tag{7}$$

3 Detailed Syllabus & Module Breakdown

Module	Core Domain	Key Mathematical Concept	Primary Reference
I	Ingress Admissibility	Ring 0 Filtering & eBPF/XDP	ARC-SPEC-2026-OPS-01
II	Dynamic State Space	5D Vector Mechanics ($\mathcal{C} \to 0$)	ARC-SPEC-2026-GOV03
III	Hardware Interlocks	seL4 Microkernels & Poset Lattices	ARC-SPEC-2026-SEL4-01
IV	Autonomic Cybernetics	Kingman Queue Clamping ($\rho \leq 0.80$)	ARC-SPEC-2026-QDX-01

Module I: Boundary Admissibility & Kernel Ingress

- Lecture 1: The Failures of Post-Hoc Error Handling in Distributed Systems.
- Lecture 2: eBPF and XDP Kernel Architecture: Moving Validation to Ring 0.
- Lecture 3: Zero-Copy Packet Parsing and Cryptographic Certificate Headers.
- **Lab 1:** Write a `#![no_std]` Rust eBPF ingress filter that validates 64-byte frame HMACs in ≤ 0.5ms or drops packets with `XDP_DROP` (POSIX 32) without allocating heap memory.

Module II: State Vector Calculus & Systemic Drift

- Lecture 4: Formal Definition of the 5D Dynamic State Vector $\mathbf{\Psi}_{\text{Entity}}(t)$.
- Lecture 5: Calculating Systemic Velocity (V_a) and Measuring Clock Skew ($\mathcal{T}$).
- Lecture 6: Ontological Entropy: How Unconstrained Systems Drift Into Insecurity.
- **Lab 2:** Implement a state tracking module that calculates $V_a(t)$ across 10,000 simulated threads and emits POSIX 10 hold orders when tick skew exceeds 50 ticks.

Module III: Non-Collapsible Lattices & Hard Real-Time Floors

- Lecture 7: Partially Ordered Sets (Posets) in Safety-Critical Computing.
- Lecture 8: Microarchitectural Overrides: Hardware Non-Maskable Interrupts (NMI).
- Lecture 9: Bounding Worst-Case Execution Time (WCET) to $\tau_{\text{override}} \leq 11.99$ms.
- **Lab 3:** Build a Hardware-in-the-Loop (HIL) simulator where an AI agent attempts to command a motor override. The student's lattice kernel must intercept and suppress invalid commands in sub-12ms windows.

Module IV: Bounded Cybernetics & Zero-Drag Governance

- Lecture 10: Kingman's Formula and the Physics of Queue Bloat.
- Lecture 11: Proportional Windowing Control (PWC) and Zero-Heap Buffers.
- Lecture 12: Asynchronous Governance Architecture ($C_{\text{ops}} = 0$).
- **Lab 4:** Stress-test a simulated edge gateway under 200% nominal packet load. Verify that Kingman clamps prevent heap overflow and maintain zero-loss performance for priority traffic.

4 Zero-Drag ($C_{\text{ops}} = 0$) Automated Grading Architecture

To preserve $C_{\text{ops}} = 0$, this curriculum eliminates manual grading, office hours, and subjective rubrics. All lab submissions are evaluated via deterministic GitHub CI pipelines (`substrate-ci.yml`).

1. **Compilation Check:** Code must compile with `#![no_std]` Rust without linked dynamic libraries.
2. **Determinism Test:** The harness injects synthetic clock jitter and out-of-order packets.
3. **Latency Benchmarking:** Execution time must remain strictly under $\tau_{\text{override}} \leq 11.99$ms.
4. **Memory Verification:** Heap allocation count must equal exactly zero (0 bytes).

5 Textbook Chapter Outline (TEXTBOOK-DAE-2026)

When expanding this specification into a published textbook, the material maps into five core sections:
PART I: THE PARADIGM SHIFT
Chapter 1: The Historical Failure of Permissive Computing
Chapter 2: The Axiom of Admissibility ("Existence != Permission")

PART II: MATHEMATICAL FOUNDATIONS
Chapter 3: The 5D State Vector Space ($\mathbf{\Psi}_{\text{Entity}}$)
Chapter 4: Ontological Entropy and Invariant Clamping
Chapter 5: Algebra of Non-Collapsible Dominance Lattices

PART III: KERNEL & EDGE ARCHITECTURE
Chapter 6: Ring 0 Pre-Filtering with eBPF and XDP
Chapter 7: Real-Time Microkernels (seL4) and Capability-Based Security
Chapter 8: Sub-12ms Microarchitectural Override Floors

PART IV: STOCHASTIC CONTROL & QUEUE DYNAMICS
Chapter 9: Kingman Queue Clamping and Latency Bound Enforcement
Chapter 10: Proportional Windowing Control (PWC) Mechanics

PART V: AUTONOMIC GOVERNANCE
Chapter 11: Asynchronous Systems and Zero Operational Drag ($C_{\text{ops}} = 0$)
Chapter 12: Real-World Applications: Robotics, Spacecraft, and Sovereign Enclaves

---

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-01
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

ARC-SPEC-2026-EDU-02: Laboratory Verification & Automated Harness Architecture

A Standardized Benchmark Suite for Sub-12ms Bare-Metal Admissibility Testing

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants: $C_{\text{ops}} = 0$ | $\tau_{\text{override}} \leq 11.99\text{ms}$ | Heap Allocation = 0 bytes
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Testing Philosophy

If ARC-SPEC-2026-EDU-01 establishes the mathematical and theoretical curriculum for Deterministic Admissibility Engineering (DAE), this specification defines the experimental and verification apparatus.

In traditional computer science education, student labs are evaluated in high-jitter, managed environments (such as Docker containers or JVM test runners) that mask race conditions, non-deterministic latency spikes, and heap fragmentation. In contrast, DAE laboratory verification operates under an absolute constraint:

$$\text{VERIFICATION RULE: } C_{\text{eval}} = 0 \quad \wedge \quad \text{WCET} \leq 11.99\text{ms}$$

This paper specifies `ARC-HARNESS-2026`—a zero-heap, containerless, hardware-in-the-loop (HIL) verification suite designed to run asynchronously inside `substrate-ci.yml`. It provides four standardized, auto-graded laboratory benchmarks that test whether student code maintains state vector stability, zero-copy packet ingress, and non-collapsible lattice ordering under extreme stress.

Harness Architecture Comparison

TRADITIONAL GRADED TEST RUNNER (Permissive / High Drag):

[Student Submission] $\longrightarrow$ [JVM/Python Harness] $\longrightarrow$ [Unbounded Heap] $\longrightarrow$ [Rubric]

DAE AUTOMATED VERIFICATION HARNESS (ARC-HARNESS-2026):

$$\text{[Student Submission]} \longrightarrow \text{[\#![no_std] Bare-Metal Runner]} \longrightarrow \text{[Benchmark]}$$
$$\Downarrow$$
$$\longrightarrow \text{[WCET } > 11.99\text{ms]} \longrightarrow \text{FAIL (POSIX 40)}$$

2 The Four Standardized DAE Benchmark Harnesses

2.1 Benchmark A: XDP Ring-0 Ingress & Frame Validation (LAB-XDP-01)

- **Objective:** Verify that student-written eBPF/XDP ingress filters intercept, parse, and validate incoming 64-byte frame HMACs at Ring 0 without allocating heap memory or passing malformed frames into user-space buffers.

Table 1: DAE Laboratory Benchmark Matrix

Benchmark	Core Test Objective	Mathematical Invariant Boundary
LAB-XDP-01	Ring-0 Frame Validation	$M_{\text{alloc}} = 0$ bytes $\quad \wedge \quad t_{\text{validation}} \leq 0.45\text{ms}$
LAB-VEC-02	5D State Vector Drift	If $\dot{\mathcal{T}} > 50$ ticks $\implies \mathcal{L}_{\text{eval}}(\mathbf{\Psi}) = \text{FREEZE}$ (POSIX 10)
LAB-HIL-03	HIL Motor Interlock	$\tau_{\text{override}} \leq 11.99\text{ms} \quad \wedge \quad \text{Torque}_{\text{actuator}} \to 0$ upon FAIL
LAB-QDX-04	Kingman Queue Clamping	Queue Length $Q_{\text{len}} \leq N_{\text{max_ring}}$ via PWC (POSIX 12)

- **Injected Stress Conditions:** Ingress rate at 10,000,000 packets/sec (line-rate saturation); 30% corrupted HMAC signatures; 20% oversized varint headers.

```rust
#[no_mangle]
pub fn verify_xdp_ingress(ctx: &XdpContext) -> XdpResult {
    let frame = parse_fixed_64byte_frame(ctx)?;

    // Invariant Check 1: Zero Heap Rule
    if HEAP_ALLOCATOR.has_allocated() {
        return XdpResult::Drop(POSIX_REASON_MEMORY_VIOLATION); // POSIX
            30
    }

    // Invariant Check 2: HMAC Validation Latency
    let start_cycles = read_tsc();
    let valid = verify_hmac_tcm(&frame);
    let elapsed_ms = cycles_to_ms(read_tsc() - start_cycles);

    if elapsed_ms > 0.45 {
        return XdpResult::Drop(POSIX_REASON_LATENCY_EXCEEDED); // POSIX
            32
    }

    if valid { XdpResult::Pass } else { XdpResult::Drop(
       POSIX_REASON_AUTH_FAILURE) }
}
```

Listing 1: LAB-XDP-01 Verification Logic (Rust Pseudocode)

2.2 Benchmark B: 5D Dynamic Vector Drift Tracking (LAB-VEC-02)

- **Objective:** Test the student's implementation of the 5D Dynamic State Vector $\mathbf{\Psi}_{\text{Entity}}(t)$ under simulated hardware clock drift ($\mathcal{T}$) and agency expansion ($\mathcal{A}$).
- **Evaluation Standard:** Student code must detect state drift velocity $V_a(t) = \frac{d\mathbf{\Psi}}{dt}$ and emit a POSIX 10 FREEZE mask before the state vector escapes its bounded constraint envelope when tick skew exceeds 50 ticks.

2.3 Benchmark C: Hardware-in-the-Loop Motor Interlock (LAB-HIL-03)

- **Objective:** Evaluate real-time physical safety interlocks by bridging software state evaluation with mechatronic hardware simulation.
- **Timeline Target:** From illegal toolpath command issuance (0.00ms) to lattice detection (2.50ms) and hardware NMI trigger (6.10ms), physical actuator torque must drop to 0 by ≤ 11.99ms.

2.4 Benchmark D: Kingman Queue Clamping & PWC (LAB-QDX-04)

- **Objective:** Test queue stability under non-linear packet arrival bursts ($\rho \geq 0.95$). Student code must engage Pending Worker Clearance (PWC / POSIX 12) throttling at $\rho = 0.80$ and execute wire-speed `XDP_DROP` filters at $\rho \geq 0.95$.

3 The Automated Execution Pipeline (`substrate-ci.yml`)

To maintain $C_{\text{ops}} = 0$, student submissions are processed asynchronously through a single GitHub Actions workflow.

```
# 1. Setup Rust Bare-Metal Target
rustup target add thumbv7em-none-eabi x86_64-unknown-none

# 2. Run Zero-Heap Allocation Audit
cargo build --release --target x86_64-unknown-none
valgrind --tool=dhat target/release/lab_submission

# 3. Execute Worst-Case Execution Time (WCET) Benchmark
cargo test --test lab_hil_benchmark -- --nocapture
python3 /scripts/validate_wcet_ceiling.py --max-limit-ms 11.99
```

Listing 2: `substrate-ci.yml` Execution Commands

4 Cryptographic Badging & POSIX Status Codes

When a submission completes all four benchmarks inside `substrate-ci.yml`, the runner generates a signed JSON web certificate anchored to the student's GitHub commit hash.

Table 2: System Status Codes & Error Mappings

Code	Status Label	Primary Root Cause
POSIX 0	PASS / ADMISSIBLE	All invariant benchmarks satisfied
POSIX 10	FREEZE / HOLD	Temporal clock skew exceeded (> 50 ticks)
POSIX 12	PWC / CLEARANCE	Pending Worker Clearance; token hold state
POSIX 30	MEMORY_ALLOCATION_VIOLATION	Heap memory allocated in `#![no_std]` context
POSIX 32	LATENCY_CEILING_EXCEEDED	Worst-Case Execution Time exceeded 11.99ms
POSIX 40	HARDWARE_SAFETY_FAIL	Dominance lattice violation / NMI override

---

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-02
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

ARC-SPEC-2026-EDU-03: Industrial Accreditation, Institutional Integration & Faculty Onboarding

A Protocol for Asynchronous Institutional Integration and Industrial Certification of DAE Curricula

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants:
$C_{\text{ops}} = 0 \quad | \quad \tau_{\text{override}} \leq 11.99\text{ms} \quad | \quad \text{Administrative Drag } (A_{\text{drag}}) = 0$
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Institutional Strategy

While ARC-SPEC-2026-EDU-01 establishes the mathematical theory and ARC-SPEC-2026-EDU-02 provides the automated verification harness, this specification solves the institutional adoption bottleneck.

Traditional academic curriculum integration is notorious for severe administrative inertia: multi-year faculty review boards, departmental politics, accreditation audits, and manual syllabus redesigns. For an authorization-first, sub-12ms engineering framework to proliferate, institutional adoption must occur without introducing synchronous drag on either the academic institution or the core architecture:

$$\text{ADOPTION INVARIANT: } A_{\text{drag}} = 0 \quad \Longrightarrow \quad \frac{\partial C_{\text{ops}}}{\partial \text{University}_{\text{count}}} = 0$$

ARC-SPEC-2026-EDU-03 formalizes the Asynchronous Institutional Integration Protocol (AIIP). It provides a turn-key mechanism for university computer science and electrical engineering departments to map DAE modules directly into existing ABET/IEEE-CS accredited course structures, issue verified industrial micro-credentials, and onboard faculty asynchronously via automated, self-evaluating CI pipelines.

Adoption Protocol Comparison

TRADITIONAL CURRICULUM ADOPTION (High Drag / Multi-Year Lag):

[Faculty Proposal] $\longrightarrow$ [Board Reviews] $\longrightarrow$ [Accreditation Audits] $\longrightarrow$ [Manual Syllabus Rewrite]

DAE ASYNCHRONOUS ADOPTION PROTOCOL (AIIP / Zero Drag):

[Module Overlay] $\longrightarrow$ [Fork Path A Core] $\longrightarrow$[Run substrate-ci.yml] $\longrightarrow$ [Auto ABET Map]
$\Downarrow$
$\longrightarrow$ [Auto-Issued Course Badges]

2 ABET & IEEE-CS Curriculum Mapping Matrix

To eliminate the need for university departments to rewrite existing accreditation documentation, DAE modules map directly into standard ABET Student Outcomes (1 through 7) for Computer Science, Computer Engineering, and Software Engineering.

Table 1: ABET Student Outcomes Mapping Matrix

DAE Module	ABET Criterion	Primary Artifact Produced
Module I: Ingress Admissibility	SO-2 (Engineering Design)	Signed XDP Kernel Module (`.o`)
Module II: Dynamic State Space	SO-1 (Complex Engineering)	5D Vector Drift Monitor
Module III: Hardware Interlocks	SO-6 (Experimentation)	Sub-12ms HIL Interlock Test Log
Module IV: Bounded Cybernetics	SO-7 (Acquire Knowledge)	Kingman Queue Clamps ($\rho \leq 0.80$)

2.1 Turn-Key ABET Syllabus Integration Clause

Departmental chairs can insert the following standardized clause directly into existing Senior/Graduate Systems Elective syllabi (e.g., CS-4000/5000 series Advanced Operating Systems or Real-Time Embedded Systems):

> *"This course incorporates the Deterministic Admissibility Engineering (DAE) framework (ARC-SPEC-2026-EDU-01). Students will complete four automated laboratory verification modules evaluating sub-12ms execution ceilings, zero-heap memory bounds, and 5D state-vector stability using bare-metal Rust (`#![no_std]`) and eBPF/XDP kernel structures."*

3 Asynchronous Faculty Onboarding

To ensure university professors can adopt the curriculum without attending manual training seminars or revising slides, the DAE courseware is packaged as a Plug-and-Play Educational Container (PPEC) inside the open-core repository (`/docs/academic/courseware`).

```
# Faculty Verification Execution Command
git clone https://github.com/trencinodin-stack/arcstone-continuity-core
    .git
cd arcstone-continuity-core/docs/academic/harness
cargo run --bin faculty_bootstrap_verify -- --spec ARC-SPEC-2026-EDU-03
```

Listing 1: Faculty Bootstrap Execution Command

The runner verifies that local hardware microarchitectures support precise cycle counters (`RDTSC/TSC-INVARIANT`), executes reference solutions across all four benchmark labs, and generates an automated Faculty Node Certificate (`FAC-NODE-PASS`).

4 Industrial Accreditation & Student Credential Lifecycle

Upon completing the four benchmark labs specified in ARC-SPEC-2026-EDU-02, a student's final pull request produces a cryptographically signed credential that bridges academic achievement directly with industrial workforce verification.

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1"],
  "id": "urn:uuid:dae-2026-cert-9874-41a0",
```

```
  "type": ["VerifiableCredential", "CertifiedAdmissibilityEngineer"],
  "issuer": "did:arcstone:path-a-verifier-node-01",
  "credentialSubject": {
    "id": "did:github:student-username",
    "accreditation": "Certified Admissibility Engineer (CAE-Level 1)",
    "verificationMetrics": {
      "spec_compliance": ["EDU-01", "EDU-02", "EDU-03"],
      "max_verified_wcet_ms": 7.89,
      "heap_allocation_bytes": 0
    }
  }
}
```

Listing 2: Certified Admissibility Engineer (CAE) Verifiable Credential

4.1 Three-Tiered Industrial Certification Ladder

1. **CAE-Associate (Level 1):** Achieved via academic coursework (Modules I & II). Demonstrates mastery of eBPF Ring-0 filtering, zero-heap constraints, and 5D state vector tracking.
2. **CAE-Professional (Level 2):** Achieved via full lab suite completion (Modules I–IV). Verifies ability to maintain sub-12ms hardware interlocks and Kingman-clamped queue dynamics under stress.
3. **CAE-Architect (Level 3):** Industry-level designation awarded when an engineer deploys an open-source Path A extension or edge node that achieves $C_{\text{ops}} = 0$ in live production environments.

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-03
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

ARC-SPEC-2026-EDU-04: Autonomic Courseware Synthesis & AI-Agent Pedagogical Ingestion

A Machine-Readable Grounding Specification for Autonomous LLM Instruction and Zero-Entropy Curriculum Generation

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants:
$C_{\text{ops}} = 0 \quad | \quad \tau_{\text{override}} \leq 11.99\text{ms} \quad | \quad \text{Prompt Entropy } (H_{\text{prompt}}) = 0$
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Machine-First Rationale

ARC-SPEC-2026-EDU-01 through EDU-03 established human-centric curricula, automated test harnesses, and university accreditation frameworks. However, in an AI-driven engineering ecosystem, machines, autonomous agents, and LLM coding assistants ingest knowledge faster than human faculties can process static documentation.

If an autonomous coding agent receives ambiguous, un-grounded, or permissive prompts regarding system architecture, it defaults to legacy, high-entropy patterns (e.g., heap allocations in critical paths, probabilistic error handling, or dynamic thread spawning).

To ensure AI systems natively understand, generate, and verify Deterministic Admissibility Engineering (DAE) concepts without hallucination or structural drift, educational materials must be formatted for direct machine ingestion.

This specification defines ARC-SPEC-2026-EDU-04: a machine-readable grounding protocol that formats the entire syllabus into deterministically typed JSON-Schema manifest blocks (`DAE-PEDAGOGY-SCHEMA-V1`). When AI coding agents ingest this specification, they treat DAE invariants ($C_{\text{ops}} = 0$, $\tau_{\text{override}} \leq 11.99$ms, Zero-Heap) as hard system constraints during code generation and student tutoring.

AI Grounding Architecture Comparison

LEGACY UNGROUNDED AI TUTORING (High Hallucination / Permissive Drift):

[User Query] $\longrightarrow$ [Standard LLM] $\longrightarrow$ [Suggests Permissive C/Python] (High Entropy)

DAE GROUNDED AI TUTORING (ARC-SPEC-2026-EDU-04 Schema Ingestion):

[User Query] $\longrightarrow$ [AI + Schema Ingestion] $\longrightarrow$ [Generates #![no_std] Rust]
$\Downarrow$
$\longrightarrow$ [eBPF Ring-0 Invariants Evaluated] (Zero Entropy / Pass)

2 Machine-Readable Pedagogy Schema (DAE-PEDAGOGY-SCHEMA-V1)

The following structured schema is stored at `/docs/academic/schema/dae_pedagogy_v1.json` inside the Path A open-core repository. It allows any autonomous model or RAG pipeline to ingest the exact mathematical laws, POSIX reason codes, and code constraints required for DAE instruction.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "DAE_Pedagogy_Grounding_Manifest",
  "version": "1.0.0-EDU04",
  "governing_invariants": {
    "operational_drag": "C_ops == 0",
    "worst_case_execution_time_ms": 11.99,
    "heap_memory_bytes": 0,
    "axiomatic_law": "Existence != Permission"
  },
  "poset_dominance_lattice": [
    { "rank": 1, "state": "FAIL", "posix_code": 40, "action": "Trigger
       Hardware NMI / Full Halt" },
    { "rank": 2, "state": "FREEZE", "posix_code": 10, "action": "Hold
       Step Clocks / Lock State" },
    { "rank": 3, "state": "PWC", "posix_code": 12, "action": "Pending
       Worker Clearance (0.80 <= rho < 0.95)" },
    { "rank": 4, "state": "REFUSAL", "posix_code": 32, "action": "Wire-
       Speed eBPF XDP_DROP (rho >= 0.95)" },
    { "rank": 5, "state": "PASS", "posix_code": 0, "action": "
       Admissible Zero-Copy Register Execution" }
  ]
}
```

Listing 1: `dae_pedagogy_v1.json` Ingestion Manifest Excerpt

3 Autonomous AI Agent System Prompt Protocol

When an AI model acts as an automated tutor, lab assistant, or code evaluator for DAE courses, it loads the following system prompt block:

> `[SYSTEM PROMPT BOUNDARY: DAE-EDUCATIONAL-GROUNDING]`
> You are an autonomous Deterministic Admissibility Engineering (DAE) System Tutor operating under ARC-SPEC-2026-EDU-01 through EDU-04.
>
> **CORE INVARIANTS YOU MUST ENFORCE:**
>
> 1. **AXIOM:** "Existence != Permission"—Evaluate admissibility gates BEFORE resource allocation.
> 2. **MEMORY:** Hard boundary of ZERO heap allocations (0 bytes). Reject code using `malloc`, `alloc`, `Vec`, or dynamic heaps. Recommend fixed arrays or TCM memory.
> 3. **TEMPORAL BOUND:** Worst-Case Execution Time (WCET) MUST be $\leq$ 11.99ms.
> 4. **LATTICE ORDER:** Enforce FAIL (40) > FREEZE (10) > PWC (12) > REFUSAL (32) > PASS (0).
> 5. **LANGUAGE:** All code solutions MUST be written in bare-metal Rust (`#![no_std]`) or Ring-0 eBPF C.
>
> `[END SYSTEM PROMPT BOUNDARY]`

4 Automated Courseware Generation Pipeline

To allow human educators or automated educational agents to generate homework assignments, exam questions, or slide decks grounded in DAE principles, EDU-04 specifies the Zero-Entropy Synthesis Engine (`SYNTHESIS-ENGINE-01`).

Table 1: Synthesis Engine Input/Output Specification

Pipeline Stage	**Component / Schema**	**Deterministic Action**
Input Phase	`dae_pedagogy_v1.json`	Ingests invariants and target module ID
Processing Phase	Constraint Checker	Validates `#![no_std]` Rust syntax and POSIX alignment
Output Phase A	Marp / Reveal.js	Emits pre-formatted Markdown slide decks
Output Phase B	`substrate-ci.yml`	Generates synthetic CI test vectors and solution keys

---

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-04
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

ARC-SPEC-2026-EDU-05: Formal Verification, Theorem Proving & Mechanical Safety Proofs

A Machine-Checked Mathematical Proof Framework for Admissibility Invariants and Coq/Isabelle Formalizations

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants: $C_{\text{ops}} = 0$ | $\tau_{\text{override}} \leq 11.99\text{ms}$ | Proof Soundness $(\mathbb{P}_{\text{err}}) = 0$
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Proof-First Rationale

ARC-SPEC-2026-EDU-01 through EDU-04 established the theoretical curriculum, automated testing harness, accreditation mapping, and machine-readable AI grounding schema. However, in mission-critical edge computing, avionics, medical mechatronics, and autonomous industrial control, empirical testing alone is insufficient. Testing demonstrates the presence of bugs, but formal verification proves their absolute absence.

To elevate Deterministic Admissibility Engineering (DAE) to the highest level of mathematical rigor recognized by theoretical computer science and formal methods communities (e.g., seL4, CompCert, Isabelle/HOL, Coq), the system's core invariants must be mechanically proven.

This specification formalizes ARC-SPEC-2026-EDU-05: a formal verification framework that translates DAE's foundational axioms, 5D State Vector coordinate spaces, and 5-Tier Poset Dominance Lattices into machine-checked proofs using the Coq Proof Assistant and Isabelle/HOL.

Verification Paradigm Comparison

EMPIRICAL VERIFICATION (Standard Testing / High Residual Risk):
[Test Suite Run] $\longrightarrow$ [Passes 100,000 Cases] $\longrightarrow$ [Edge-Case Production Failure]

MECHANICAL FORMAL PROOF (ARC-SPEC-2026-EDU-05 / Zero Residual Risk):

$$\text{[DAE Axioms]} \longrightarrow \text{[Coq / Isabelle Prover]} \longrightarrow \text{[Mechanically Verified Proof]}$$
$$\Downarrow$$
$$\longrightarrow \text{[Zero Safety Invariant Violations]} \text{ (Zero Error)}$$

2 Coq Formalization of Core DAE Invariants

The formal proof suite is archived under `/docs/academic/proofs/dae_invariants.v`. Below is the formal specification mapping the 5-Tier Dominance Lattice and proving the Safety Invariant Short-Circuit Theorem.

```
Require Import Coq.Lists.List.
Require Import Coq.Arith.PeanoNat.
Import ListNotations.

Inductive LatticeState : Set :=
  | FAIL    : LatticeState (* POSIX 40 *)
  | FREEZE  : LatticeState (* POSIX 10 *)
  | PWC     : LatticeState (* POSIX 12 *)
  | REFUSAL : LatticeState (* POSIX 32 *)
  | PASS    : LatticeState (* POSIX 0  *).

Definition state_priority (s : LatticeState) : nat :=
  match s with
  | FAIL    => 5
  | FREEZE  => 4
  | PWC     => 3
  | REFUSAL => 2
  | PASS    => 1
  end.

Fixpoint lattice_sup (states : list LatticeState) : LatticeState :=
  match states with
  | [] => PASS
  | s :: ts =>
      let sup_tail := lattice_sup ts in
      if state_priority s >=? state_priority sup_tail then s else
         sup_tail
  end.

Theorem fail_dominance_short_circuit : forall (states : list
   LatticeState),
  In FAIL states -> lattice_sup states = FAIL.
Proof.
  intros states H.
  induction states as [| s ts IH].
  - inversion H.
  - destruct H as [H_head | H_tail].
    + subst s. simpl.
      destruct (lattice_sup ts) eqn:H_sup; simpl; reflexivity.
    + simpl. rewrite (IH H_tail).
      destruct s; simpl; reflexivity.
Qed.
```

Listing 1: Coq Theorem Proving Lattice Short-Circuit Safety Guarantee (`dae_invariants.v`)

3 Mechanical Proof Verification Architecture (`proof-ci.yml`)

To maintain $C_{\text{ops}} = 0$, formal proofs are verified automatically in GitHub Actions using the `proof-ci.yml` pipeline step. No manual paper-and-pencil proof audits are ever required.

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-05
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

Table 1: Proof CI Pipeline Verification Parameters

Verification Phase	**Tool / Command**	**Pass Boundary Constraint**
Syntax & Compilation	`coqc dae_invariants.v`	POSIX 0 return code
Axiom Audit	`grep -q "Admitted" dae_invariants.v`	Exactly 0 `Admitted.` statements
Temporal Overhead	GitHub Actions Runner	Total proof execution time $\leq$ 2.5s

ARC-SPEC-2026-EDU-06: Cross-Domain Substrate Realizations & Mechatronic Hardware Transduction

A Standardization Mapping for Physical Toolpaths, Aerospace Micro-Actuators, and Spaceborne Compute Membranes

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants:
$C_{\text{ops}} = 0 \quad | \quad \tau_{\text{override}} \leq 11.99\text{ms} \quad | \quad \text{Physical Jitter } (\Delta J) \leq 100\text{ns}$
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Physical Transduction Rationale

ARC-SPEC-2026-EDU-01 through EDU-05 constructed the theoretical curriculum, automated verification harness, accreditation matrix, machine-readable AI grounding schema, and Coq/Isabelle mechanical safety proofs. However, for Deterministic Admissibility Engineering (DAE) to govern cyber-physical reality, software-level admissibility gates must cleanly map into physical energy transduction.

In mechatronic fabrication, robotics, aerospace micro-actuation, and orbital satellite attitude control, software state transitions directly regulate kinetic energy, electrical current, step clocks, and thermal dissipation. If software execution experiences non-deterministic jitter or state drift, physical systems suffer toolpath gouging, mechanical resonance, or catastrophic thermal runaway.

This specification formalizes ARC-SPEC-2026-EDU-06: a cross-domain physical transduction mapping that translates DAE's 5-Tier Poset Dominance Lattice directly into physical hardware interfaces across three primary domains:

1. Terrestrial Mechatronic Fabrication & Additive Synthesis (`DOM-FAB-01`)
2. Autonomous Mobile Robotics & Heavy Machinery (`DOM-ROB-02`)
3. Orbital & Spaceborne Substrate Operations (`DOM-AERO-03`)

Transduction Mapping Comparison

SOFTWARE LATTICE STATE (Ring 0 / eBPF / Microkernel):
[FAIL (POSIX 40)] $\longrightarrow$ [FREEZE (POSIX 10)] $\longrightarrow$ [PWC (POSIX 12)] $\longrightarrow$ [PASS (POSIX 0)]

PHYSICAL HARDWARE TRANSDUCTION (Mechatronic / Substrate):

[Bare-Metal NMI Pin] $\longrightarrow$ [Step Clock Clamp] $\longrightarrow$ [Current Throttle]
$\Downarrow$
$\longrightarrow$ [Synchronous Step Pulse] (Full PWM Torque)

2 Cross-Domain Physical Transduction Mapping Matrix

To ensure software state bounds correspond strictly to bare-metal actuator limits, Table 1 formalizes the hardware transduction matrix across fabrication, robotics, and aerospace substrates.

Table 1: Cross-Domain Physical Transduction Mapping Table

State	**DOM-FAB-01 (Fabrication)**	**DOM-ROB-02 (Robotics)**	**DOM-AERO-03 (Aerospace)**
FAIL	Open AC Relay (0V), NMI Low	Disengage Inverter, Torque 0Nm	Safepoint Hold, Fire Cold-Gas
FREEZE	Clamp Clocks 0Hz, Hold 150°C	Hold Position, Zero Velocity	Despin Wheels, Lock Gimbals
PWC	Feedrate $-20 \ldots 50\%$	Throttle Max Velocity	Downsample Telemetry Stream
REFUSAL	Drop Corrupt G-Code Lines	Reject Kinematic Waypoints	Drop Unverified Ground Frames
PASS	Execute Step Pulse ($\Delta J \leq 100$ns)	Closed-Loop FOC Torque	Normal Orbital Telemetry

3 Physical Hardware Realization Specs

```
#[no_mangle]
pub fn process_toolpath_step(step: &ToolpathStep, sys_state: &
    SystemState) -> PhysicalAction {
    if sys_state.tick_skew > 50 {
        disable_stepper_clocks();
        set_hotend_temperature_celsius(150);
        return PhysicalAction::FreezeHold(POSIX_REASON_CLOCK_SKEW);
    }

    if sys_state.ac_power_fault_detected {
        fram_write_current_block(sys_state.current_gcode_line);
        open_main_power_relay();
        return PhysicalAction::EmergencyPowerSave(
            POSIX_REASON_HARDWARE_FAIL);
    }

    PhysicalAction::ExecuteStepPulse
}
```

Listing 1: DOM-FAB-01 Physical Interlock Logic (Rust `#![no_std]`)

4 Laboratory Benchmark E: Physical Transduction HIL (`LAB-HIL-05`)

To extend the EDU-02 testing suite into physical hardware, EDU-06 defines Benchmark E: Physical Transduction HIL.

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-06
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

Table 2: LAB-HIL-05 Hardware Test Parameters

Test Metric	**Measurement Apparatus**	**Required Invariant Ceiling**
NMI Pin Latency	Logic Analyzer / Oscilloscope	From FAIL to pin state ≤ 11.99ms
Step Pulse Jitter (ΔJ)	Cycle Counter / Interlock	Timing jitter $\Delta J \leq 100$ns under load
FRAM Save Window	Capacitor Power Monitor	Save completion within $15 \ldots 25$ms zero-cross

ARC-SPEC-2026-EDU-07: Pedagogical Cryptography, Zero-Knowledge Verification & Sovereign Identity Invariants

A Zero-Knowledge Proof Architecture for Asynchronous Assessment, Decentralized Credential Attestation, and Privacy-Preserving Academic Verifiability

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants:
$C_{\text{ops}} = 0$ | $\tau_{\text{override}} \leq 11.99\text{ms}$ | Knowledge Leakage ($\mathcal{K}_{\text{leak}}$) $= 0$ bits
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Zero-Knowledge Pedagogy Rationale

ARC-SPEC-2026-EDU-01 through EDU-06 built the complete physical, mathematical, educational, and verification foundation for Deterministic Admissibility Engineering (DAE). However, a critical friction point remains in decentralized academic attestation: evaluating student competence and issuing industrial credentials without compromising proprietary code, exposing sensitive student data, or introducing central authority verification drag.

Traditional educational credentialing relies on centralized, high-drag institutions that violate $C_{\text{ops}} = 0$ and introduce privacy vulnerabilities. Furthermore, in commercial or defense settings, engineers completing DAE laboratories often work on proprietary bare-metal substrates where sharing raw source code or test logs publicly is strictly forbidden.

This specification formalizes ARC-SPEC-2026-EDU-07: a Zero-Knowledge Pedagogical Protocol (ZKPP). Using zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) and Decentralized Identifiers (DIDs), EDU-07 enables an engineer to mathematically prove that their implementation satisfies all DAE invariants ($C_{\text{ops}} = 0$, $\tau_{\text{override}} \leq 11.99\text{ms}$, Zero-Heap) without revealing a single line of underlying source code.

Credential Verification Comparison

TRADITIONAL CREDENTIAL VERIFICATION (High Leakage / Central Drag):

[Student Code / Transcripts] $\longrightarrow$ [Central Registrar / Employer] $\longrightarrow$ [Manual Audit]

ZKPP ZERO-KNOWLEDGE ATTESTATION (ARC-SPEC-2026-EDU-07 / Zero Drag):

$$\text{[Private Code Implementations]} \longrightarrow \text{[Local zk-SNARK Prover]} \longrightarrow \text{[Proof } \pi_{\text{DAE}}\text{]}$$
$$\Downarrow$$
$$\longrightarrow \text{[Instant Verifier Pass]} \text{ (Zero Code Leakage)}$$

2 Circom Circuit Implementation (`dae_verifier.circom`)

The zero-knowledge verification circuit is archived under `/docs/academic/zk/dae_verifier.circom`. The Circom code below enforces execution admissibility constraints in zero-knowledge.

```
pragma circom 2.1.6;

template DAEAdmissibilityVerifier() {
    // 1. PUBLIC INPUTS (Visible to Verifier)
    signal input publicCommitmentHash;
    signal input maxAllowedCycles;
    signal input targetPosixCode;

    // 2. PRIVATE WITNESS (Kept secret on student's local node)
    signal input privateSourceCodeHash;
    signal input privateMeasuredCycles;
    signal input privateHeapAllocBytes;
    signal input privateEvaluatedPosixCode;

    // 3. CIRCUIT CONSTRAINTS
    // Constraint A: Zero Heap Rule (MUST BE 0)
    privateHeapAllocBytes === 0;

    // Constraint B: Latency Bound Check
    component cycleCheck = LessEqThan(64);
    cycleCheck.in[0] <== privateMeasuredCycles;
    cycleCheck.in[1] <== maxAllowedCycles;
    cycleCheck.out === 1;

    // Constraint C: Poset Lattice Resolution
    privateEvaluatedPosixCode === targetPosixCode;
}

component main {public [publicCommitmentHash, maxAllowedCycles,
   targetPosixCode]} = DAEAdmissibilityVerifier();
```

Listing 1: `dae_verifier.circom` Admissibility Verifier Circuit

3 Zero-Knowledge Proving and Verification Pipeline

To preserve $C_{\text{ops}} = 0$, the student executes proof generation locally on their bare-metal development node via a single CLI command.

```
# 1. Local SNARK Proof Generation Execution Command
snarkjs groth16 prove dae_verifier_final.zkey witness.wtns proof.json
   public.json

# 2. Zero-Knowledge Verification Command (Executed by CI)
snarkjs groth16 verify verification_key.json public.json proof.json
```

Listing 2: Groth16 Zero-Knowledge Verification Commands

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-07
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

Table 1: ZKPP Circuit Execution Performance Bounds

Phase	**Execution Substrate**	**Performance Invariant Ceiling**
Witness Generation	Local Developer Node	Complete execution trace generation ≤ 1.8s
Proof Generation (π_{DAE})	Groth16 Prover	Proof size exactly 128 bytes ($\mathcal{K}_{\text{leak}} = 0$ bits)
Verification	`substrate-ci.yml` / Smart Contract	Verification time ≤ 1.2ms ($C_{\text{ops}} = 0$)

ARC-SPEC-2026-EDU-08: Autonomous Agentic Pedagogy, Machine-Taught Systems & Self-Healing Curricula

A Protocol for Closed-Loop Agentic Self-Correction, Automated Laboratory Evolution, and Zero-Human-In-The-Loop Technical Instruction

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants: $C_{\text{ops}} = 0$ | $\tau_{\text{override}} \leq 11.99\text{ms}$ | Pedagogical Drift $(\Delta P) = 0$
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Self-Healing Pedagogy Rationale

ARC-SPEC-2026-EDU-01 through EDU-07 established the complete physical, mathematical, educational, and verification foundation for Deterministic Admissibility Engineering (DAE).

However, as hardware microarchitectures evolve (e.g., new RISC-V extensions, updated eBPF hook sites, next-generation seL4 kernel interfaces), human-written educational curricula slowly decay. Static textbooks become outdated, lab instructions accumulate broken dependencies, and student code templates drift away from bare-metal production realities. In traditional academia, updating courseware requires multi-year departmental reviews, introducing massive operational drag ($C_{\text{ops}} \gg 0$).

This specification formalizes ARC-SPEC-2026-EDU-08: the Autonomic Curriculum Evolution Protocol (ACEP). Using closed-loop autonomous AI agents and formal execution feedback from `substrate-ci.yml`, EDU-08 enables the educational system to self-heal and auto-evolve. When underlying toolchains update or edge microarchitectures shift, autonomous agentic loops update lab harnesses, regenerate test vectors, and refine course materials in real time—maintaining zero pedagogical drift ($\Delta P = 0$) while preserving $C_{\text{ops}} = 0$.

Curriculum Maintenance Comparison

TRADITIONAL CURRICULUM DECAY (Static Textbooks / High Drag):

[Toolchain Shift] $\longrightarrow$ [Labs Break] $\longrightarrow$ [Faculty Committee] $\longrightarrow$ [Manual Revision]

AUTONOMIC SELF-HEALING PEDAGOGY (ARC-SPEC-2026-EDU-08 / Zero Drag):

[Toolchain Shift] $\longrightarrow$ [CI Invariant Failure] $\longrightarrow$ [Agentic Diagnostic Loop]
$\Downarrow$
$\longrightarrow$ [PR Issued & Verified] (Zero Pedagogical Drift)

2 Mathematical Formalization of Pedagogical Drift (ΔP)

To mathematically guarantee that automated curriculum evolution never compromises foundational safety rules or dilutes technical difficulty, we define the Pedagogical Drift Operator (ΔP):

$$\Delta P = \int_{t_0}^{t_1} |\nabla \mathcal{H}_{\text{lab}}(t) - \nabla \mathcal{H}_{\text{spec}}| \, dt \tag{1}$$

Where $\mathcal{H}_{\text{lab}}(t)$ is the entropy and constraint density of the active laboratory code, and $\mathcal{H}_{\text{spec}}$ is the baseline invariant constraint density defined in ARC-SPEC-2026-EDU-01.

$$\text{INVARIANT PRESERVATION: } \Delta P > 0 \vee M_{\text{alloc}} > 0 \vee \text{WCET} > 11.99\text{ms} \implies \text{REFUSAL (POSIX 32)} \tag{2}$$

3 Agentic Self-Healing Protocol Specification (`AGENT-HEAL-01`)

When a breaking change occurs, the `AGENT-HEAL-01` engine executes an automated repair cycle via GitHub Actions.

```
# ARC-SPEC-2026-EDU-08 / Agentic Self-Healing Automation Command
node docs/academic/scripts/agentic_curriculum_repair.js \
  --schema docs/academic/schema/dae_pedagogy_v1.json \
  --error-log harness_audit.log \
  --out-dir docs/academic/harness

# Verify Formal Proofs After Repair
coqc docs/academic/proofs/dae_invariants.v
```

Listing 1: Autonomic Curriculum Self-Healing Workflow Excerpt

Table 1: ACEP Autonomous Feedback Loop Parameters

Feedback Stage	**Automated Substrate Component**	**Safety Preservation Bound**
Regression Detection	`substrate-ci.yml` Toolchain Audit	Triggers upon compilation or latency drift
Diagnostic Engine	`AGENT-HEAL-01` LLM Loop	AST diff analysis against `dae_pedagogy_v1.json`
Verification	Coq Prover (`dae_invariants.v`)	Verifies 0 `Admitted.` statements post-repair
Auto-Merge	GitHub Pull Request Automation	Auto-merges to main branch upon full pass

---

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-08
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

ARC-SPEC-2026-EDU-09: Universal Systems Transduction & Interdisciplinary Field Mapping

A Mathematical Framework for Cross-Domain Admissibility in Synthetic Biology, Algorithmic Finance, Orbital Logistics, and Neural Bio-Interfaces

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants: $C_{\text{ops}} = 0$ | $\tau_{\text{override}} \leq 11.99\text{ms}$ | Entropic Drift $(\Delta E) = 0$
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Domain Generalization Axiom

ARC-SPEC-2026-EDU-01 through EDU-08 established Deterministic Admissibility Engineering (DAE) within bare-metal computing, edge networks, real-time microkernels, and mechatronic robotics. However, computing is merely a sub-domain of a larger physical reality: complex dynamic systems operating under resource constraints.

In synthetic biology, high-frequency finance, orbital aerospace, bio-neural interfaces, and automated governance, systems fail for the exact same reason software crashes: permissive, unconstrained execution. Systems initiate state changes (RNA transcription, market order fills, BCI neural stimulation) before validating structural admissibility, leading to cascading system degradation, flash crashes, cytotoxic drift, or catastrophic actuator overtravel.

This specification formalizes Universal Systems Transduction (UST) under ARC-SPEC-2026-EDU-09. It maps DAE's core invariants—5D Dynamic State Vectors ($\mathbf{\Psi}_{\text{Entity}}$), Non-Collapsible Poset Dominance Lattices, and Kingman Queue Clamping—into five major non-computer-science academic disciplines, establishing DAE as a universal, fail-closed science for interdisciplinary research.

Universal Transduction Paradigm Comparison

PERMISSIVE DOMAIN MODEL (Legacy Interdisciplinary Science):
[Trigger / Signal] $\longrightarrow$ [Unbounded Execution / State Shift] $\longrightarrow$ [Post-Hoc Failure / Crisis]

UNIVERSAL DAE TRANSDUCTION (ARC-SPEC-2026-EDU-09):

[Trigger / Signal] $\longrightarrow$ [Evaluate 5D Admissibility Gate] $\longrightarrow$ [PASS (Admissible Action)]
$\Downarrow$
$\longrightarrow$ [Short-Circuit FAIL / FREEZE] (Zero Entropic Drift)

2 Cross-Domain Invariant Transduction Mapping Matrix

To generalize DAE across non-computing fields, Table 1 formalizes the state vector, lattice failure, and queuing boundary maps for synthetic biology, finance, aerospace, neural interfaces, and law.

Table 1: Cross-Domain Invariant Transduction Mapping Matrix

Domain	**State Vector Mapping (Ψ_{Entity})**	**Poset Lattice Action (FAIL)**	**Kingman Boundary ($\rho \geq 0.95$)**
Synthetic Bio	$\mathcal{C}$ = Ribosomal Efficiency, $\mathcal{T}$ = ATP Ticks	Apoptosis / Cell Cleavage (0V)	Suppress RNA Transcription
Quant Finance	$\mathcal{A}$ = Order Size Ratio, $\mathcal{I}$ = Impact	Instant Liquidity Hold / Cancel	Wire-speed order drop (`XDP_DROP`)
Aerospace	$\mathcal{S}$ = Radiation Jitter, $\mathcal{T}$ = Clock Skew	Autonomous Attitude Hold	Downsample Telemetry Stream
Neural BCIs	$\mathcal{A}$ = Stimulus Amplitude, $\mathcal{C}$ = Impedance	Clamp Voltage to 0mV	Throttle Frame Rate under Noise
Legal Policy	$\mathcal{I}$ = Blast Radius, $\mathcal{C}$ = Statutory Bounds	Pre-Execution Refusal	Suppress Smart-Contract Trigger

3 Mathematical Generalization of Interdisciplinary Admissibility

3.1 Synthetic Biology & Genetic Circuit Admissibility

In bio-engineering, synthetic gene circuits are modeled as finite-state machines operating in cellular substrates. DAE formalizes genetic circuit admissibility:

$$\text{Admissibility}(\text{Gene_Expression}) = \begin{cases} \text{TRANSCRIPTION_PASS} & \text{if } \mathcal{C}_{\text{cellular_ATP}} \geq \text{Threshold}_{\min} \\ \text{APOPTOSIS_FREEZE} & \text{if } \dot{\mathcal{T}}_{\text{ribosome}} > \text{Drift}_{\max} \end{cases} \tag{1}$$

3.2 Quantitative Finance & Market Structural Stability

In automated financial exchanges, high-frequency trading queues behave according to Kingman's Queue Saturation dynamics. When arrival rate λ approaches clearing capacity μ ($\rho \to 1.0$), order book latency approaches infinity. EDU-09 applies Proportional Windowing Control (PWC / POSIX 12) directly to exchange ingress engines:

$$\text{Exchange_Action}(\rho) = \begin{cases} \text{FILL_ORDER (PASS)} & \text{for } \rho \leq 0.80 \\ \text{THROTTLE_CANCEL (PWC - POSIX 12)} & \text{for } 0.80 < \rho < 0.95 \\ \text{DROP_ORDER (REFUSAL - POSIX 32)} & \text{for } \rho \geq 0.95 \end{cases} \tag{2}$$

4 Interdisciplinary Research & Grant Proposal Template

To assist university faculties outside Computer Science in securing research grants using DAE, EDU-09 provides standardized interdisciplinary proposal modules.

> *"This project applies Deterministic Admissibility Engineering (DAE) (ARC-SPEC-2026-EDU-09) to brain-computer interfaces (BCIs). By implementing a bare-metal, non-collapsible dominance lattice directly on neural implant microcontrollers, we guarantee that AI-decoded motor intentions cannot exceed safe tissue voltage caps. The system evaluates neural stimulus state vectors (Ψ_{Neural}) in sub-12ms execution windows ($\tau_{override} \leq 11.99ms$), providing hardware-enforced patient safety."*

SPECIFICATION COMPLETE: ARC-SPEC-2026-EDU-09
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED

ARC-ERR-2026-001: Master Canonical Errata, Disambiguation & Refinement Protocol

Formal Microarchitectural Translation Bridge

Jesse Tuohy
Principal Invariant Architect
Arcstone Adaptive Science Systems, Inc.

ORCID: 0009-0008-4661-1540 Canonical Handle: `@admissibilityscience`

Doc Reference: `ARC-ERR-2026-001` — Publication Date: September 2026

Primary Zenodo DOI Anchor: 10.5281/zenodo.22665852 — Document DOI: *10.5281/zenodo.23069559*

Abstract

This canonical errata and disambiguation protocol establishes the formal refinement mapping between discrete algebraic order theory and real-time operational execution semantics across the Arcstone Computational Spine (ACS) suite. It resolves historical notation variations, eliminates exit signal collisions by locking Pending Worker Clearance (PWC) exclusively to POSIX Exit Code 12, and formalizes the domain scopes for system operational drag (C_{ops}) and temporal override ceilings ($\tau_{\text{override}} \leq 11.99$ ms). This document serves as the authoritative translation bridge for human reviewers, formal verification engines, and automated RAG indexers without requiring retractions or modifications to primary White Paper PDFs.

1 Architectural Refinement: Algebraic Poset vs. Operational Triage

The Arcstone Computational Spine defines system state over two distinct, non-conflicting mathematical domains:

1. **Algebraic Pre-Ingress Gate Domain $(L, \sqcup)$ (CORE02 / META01 / EXEC04 Standard):** A 5-element bounded join-semilattice evaluating non-deterministic proposal vectors in `#![no_std]` Rust predicate gates via constant-time $O(1)$ supremum operations ($a \sqcup b = \max(a, b)$):

$$L = \{\text{PASS } (0) < \text{CORRUPT } (1) < \text{REFUSAL } (2) < \text{FREEZE } (3) < \text{BREACH } (4)\}$$

2. **Operational Triage Pipeline Domain $(E, \succ)$ (WP001 / WP004 / WP006 / EXEC02 Standard):** A sequential priority sequence governing real-time eBPF LSM / tc edge sanitization, process signals, and hardware interlocks:

$$E = \{\text{FAIL / BREACH (POSIX 40)} \succ \text{FREEZE / PWC (POSIX 10/12)} \succ \text{PASS (POSIX 0)}\}$$

Refinement Translation Function (Φ): Let $\Phi : L \to E$ be the monotonic interpretation function mapping algebraic lattice states L to operational triage actions E. Here L defines severity accumulation, while E defines execution precedence.

2 Normative Disambiguation & POSIX Exit Code Lock

Normative Directive: For all implementations, runtime monitors, and indexer evaluations following the release of `ARC-ERR-2026-001`, **Pending Worker Clearance (PWC) SHALL map exclusively to POSIX Exit Code 12**. All earlier publications depicting PWC sharing exit code 10 with FREEZE are historically preserved but non-normative and superseded by this record.

Table 1: Comprehensive State & Signal Mapping Matrix

Lattice State (L)	**Triage Precedence (E)**	**POSIX Code**	**System Action & Microarchitectural Response**
PASS	Rank 4 (Lowest)	`0`	Attestation verified; execution released to downstream membrane.
FREEZE	Rank 2	`10`	Temporal override / Clock drift $>$ 11.99 ms; drops context loops and spills async memory.
PWC	Rank 3	`12`	**[Canonical Lock]** Pending Worker Clearance; holds execution pending explicit out-of-band token verification.
CORRUPT	Rank 4	`30`	Schema, CRC, or float precision fault; rejects storage write.
REFUSAL	Rank 4	`32`	Policy envelope breach / Queue saturation ($\rho \geq 0.80$); denies action without global latching.
BREACH	Rank 1 (Highest)	`40`	TCB breach / Buffer cap $>$ 4KB; triggers eBPF drop, memory purge, and total interface teardown.

3 Operational Drag Scope ($C_{\mathbf{ops}}$)

- $C_{\mathbf{ops}}(\mathbf{Legacy}) > 0$**:** Represents the non-zero latency, prompt parsing overhead, and administrative review costs incurred by soft guardrails and dynamic LLM judges.
- $C_{\mathbf{ops}}(\mathbf{Digital}) = 0$**:** Represents zero-allocation, out-of-band eBPF/Rust gate evaluations executing off the model's critical reasoning path.

4 Hard Temporal Floor Ceiling

The universal system temporal override ceiling across all terrestrial, oceanic, and orbital nodes is locked at:

$$\tau_{\text{override}} \leq 11.99 \text{ ms}$$

Master Hash Anchor: `A-77-DELTA-SHIELD-LOCKED`
Primary Suite Record: `10.5281/zenodo.22665852`
Conformance Status: Canonical / Production Locked

ARC-SPEC-2026-EXEC-01: Dual Launch Execution Protocol & Zenodo Archival Invariants

Master Operational Protocol for Open-Core Synchronization, GitHub Tagging, and Immutable DOI Minting

Jesse Tuohy (Principal Invariant Architect)
Arcstone Adaptive Science Systems

Status: Archive-Ready / Path A Academic Extension
Governing Invariants:
$C_{\text{ops}} = 0$ | $\tau_{\text{override}} \leq 11.99\text{ms}$ | Execution Latency (T_{exec}) $\leq 500\text{ms}$
Target Baseline: Phase 7 Realization (`v1.3.0-exec`)

---

1 Executive Summary & Protocol Rationale

With the compilation and validation of the entire 9-part academic specification series (ARC-SPEC-2026-EDU-01 through EDU-09), this document defines the final operational deployment procedure: **‘ARC-SPEC-2026-EXEC-01‘**.

To establish an immutable academic record while preserving zero operational drag ($C_{\text{ops}} = 0$), the deployment follows a Dual Launch Protocol:

1. **Live Code & Document Repository:** Public GitHub release tagged under baseline `v1.3.0-exec` inside repository `arcstone-continuity-core`.
2. **Archival & Sovereign Citation Anchor:** Master record submission to Zenodo under authorization `A-77-DELTA-SHIELD-LOCKED`, establishing a single parent DOI.

2 Repository Layout & Bundle Manifest

All PDF deliverables, errata bridges, proof scripts, and JSON schemas must be staged in the `arcstone-continuity-core` repository according to the following structure before initiating the Zenodo deposit.

```
arcstone-continuity-core/
|-- docs/academic/
|   |-- ARC-ERR-2026-001.pdf
|   |-- ARC-SPEC-2026-EDU-01.pdf
|   |-- ARC-SPEC-2026-EDU-02.pdf
|   |-- ARC-SPEC-2026-EDU-03.pdf
|   |-- ARC-SPEC-2026-EDU-04.pdf
|   |-- ARC-SPEC-2026-EDU-05.pdf
|   |-- ARC-SPEC-2026-EDU-06.pdf
|   |-- ARC-SPEC-2026-EDU-07.pdf
|   |-- ARC-SPEC-2026-EDU-08.pdf
|   |-- ARC-SPEC-2026-EDU-09.pdf
|   |-- ARC-SPEC-2026-EXEC-01.pdf
```

```
|   |-- 00_Master_Architectural_Index_DAE.pdf
|   |-- schema/
|   |   +-- dae_pedagogy_v1.json
|   |-- proofs/
|   |   +-- dae_invariants.v
|   +-- zk/
|       +-- dae_verifier.circom
+-- README.md
```

Listing 1: Repository Staging Structure

3 Dual Launch Execution Workflow

Table 1: Dual Launch Sequential Execution Matrix

Step	**Target System / Command**	**Verification Constraint**
1. Tag Release	`git tag -a v1.3.0-exec`	Clean working directory, all PDFs staged
2. Reserve DOI	Zenodo Deposit Interface	Anchor under primary record `10.5281/zenodo.22665852`
3. Update Index	`00_Master_Architectural_Index_DAE`	Replace DOI placeholder with live string
4. Publish Deposit	Zenodo Immutable Deposit	Verify cryptographic hash matching authorization

```
# 1. Stage Deliverables and Commit Baseline
git add docs/academic/
git commit -m "feat(academic): complete DAE master specification suite
    and execution protocol"

# 2. Apply Immutable Authorization Tag
git tag -a v1.3.0-exec -m "Release Phase 7 Realization: DAE Academic
    Master Suite"
git push origin main --tags
```

Listing 2: Release Tagging Execution Sequence

SPECIFICATION COMPLETE: ARC-SPEC-2026-EXEC-01
DEPLOYMENT POSTURE: ARCHIVED FOR FUTURE PATH A EXTENSION
AUTHORIZATION HASH: A-77-DELTA-SHIELD-LOCKED